



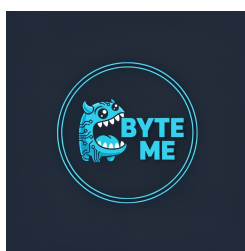
UNIVERSITÀ  
DEGLI STUDI  
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

# Piano di Qualifica

|                             |                                                 |
|-----------------------------|-------------------------------------------------|
| <b>Versione</b>             | 1.1.4                                           |
| <b>Stato</b>                | Da approvare                                    |
| <b>Redazione</b>            | Elisa Marchioro, Giulia Barzon                  |
| <b>Verifica<sub>G</sub></b> | Verificato                                      |
| <b>Approvazione</b>         | Joseph Grant                                    |
| <b>Proprietario</b>         | ByteMe                                          |
| <b>Uso</b>                  | Esterno                                         |
| <b>Destinatari</b>          | Prof. Tullio Vardanega<br>Prof. Riccardo Cardin |

## Registro dei Cambiamenti

| Versione | Data       | Autore          | Descrizione                                                                      | Verifica <sub>G</sub>             |
|----------|------------|-----------------|----------------------------------------------------------------------------------|-----------------------------------|
| 1.1.4    | 17/04/2026 | Matteo Cuogo    | Aggiunta sezione descrizione selezione modello per la correzione grammaticale    |                                   |
| 1.1.3    | 16/04/2026 | Matteo Cuogo    | Aggiunte descrizioni ad ulteriori test                                           |                                   |
| 1.1.2    | 12/04/2026 | Giulia Barzon   | Aggiunta spiegazione metriche di qualità dell'output generativo <sub>G</sub>     |                                   |
| 1.1.1    | 11/04/2026 | Elisa Marchioro | Mini fix                                                                         | Giulia Barzon e Chiara Grossele   |
| 1.1.0    | 11/04/2026 | Elisa Marchioro | Aggiunti nuovi test a unit testing e miglioramento della struttura del documento | Giulia Barzon e Chiara Grossele   |
| 1.0.2    | 07/04/2026 | Elisa Marchioro | Aggiunta sezione unit testing                                                    | Giulia Barzon                     |
| 1.0.1    | 01/04/2026 | Giulia Barzon   | Aggiornamento fonti e riferimenti                                                | Tommaso Tom-bacco                 |
| 1.0.0    | 26/02/2026 | Joseph Grant    | Approvazione finale del documento                                                |                                   |
| 0.2.0    | 24/02/2026 | Giulia Barzon   | Aggiornamento cruscotto di qualità <sub>G</sub>                                  | Elisa Marchioro e Chiara Grossele |
| 0.1.5    | 21/02/2026 | Giulia Barzon   | Aggiunti test analisi dinamica <sub>G</sub>                                      | Elisa Marchioro e Chiara Grossele |
| 0.1.4    | 05/02/2026 | Giulia Barzon   | Aggiornamento cruscotto di qualità <sub>G</sub>                                  | Elisa Marchioro e Chiara Grossele |
| 0.1.3    | 11/01/2026 | Elisa Marchioro | Aggiunte metriche qualità di prodotto                                            | Giulia Barzon                     |
| 0.1.2    | 11/01/2026 | Giulia Barzon   | Aggiunte metriche qualità di processo                                            | Elisa Marchioro                   |
| 0.1.1    | 27/12/2025 | Joseph Grant    | Aggiunta sezione 'verifica <sub>G</sub> ' al versionamento <sub>G</sub>          | Giulia Barzon                     |
| 0.1.0    | 15/12/2025 | Giulia Barzon   | Creazione del documento, scrittura introduzione e checklist <sub>G</sub>         | Elisa Marchioro                   |

Tabella 1: Registro delle versioni del documento

# Indice

|          |                                                                                                                               |           |
|----------|-------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduzione</b>                                                                                                           | <b>1</b>  |
| 1.1      | Scopo del documento . . . . .                                                                                                 | 1         |
| 1.2      | Scopo del prodotto . . . . .                                                                                                  | 1         |
| 1.3      | Glossario <sub>G</sub> . . . . .                                                                                              | 1         |
| 1.4      | Riferimenti . . . . .                                                                                                         | 1         |
| 1.4.1    | Riferimenti normativi . . . . .                                                                                               | 1         |
| 1.4.2    | Riferimenti informativi . . . . .                                                                                             | 2         |
| <b>2</b> | <b>Metriche di qualità</b>                                                                                                    | <b>3</b>  |
| 2.1      | Qualità di processo . . . . .                                                                                                 | 3         |
| 2.1.1    | Processi primari di fornitura <sub>G</sub> . . . . .                                                                          | 3         |
| 2.1.2    | Processi primari di sviluppo <sub>G</sub> . . . . .                                                                           | 6         |
| 2.1.3    | Processi di supporto <sub>G</sub> : documentazione e qualità . . . . .                                                        | 7         |
| 2.2      | Qualità di prodotto . . . . .                                                                                                 | 8         |
| 2.2.1    | Adeguatezza funzionale <sub>G</sub> . . . . .                                                                                 | 8         |
| 2.2.2    | Qualità dell'output generativo <sub>G</sub> . . . . .                                                                         | 8         |
| 2.2.3    | Affidabilità <sub>G</sub> . . . . .                                                                                           | 9         |
| 2.2.4    | Efficienza . . . . .                                                                                                          | 9         |
| 2.2.5    | Usabilità <sub>G</sub> . . . . .                                                                                              | 10        |
| 2.2.6    | Manutenibilità <sub>G</sub> . . . . .                                                                                         | 10        |
| <b>3</b> | <b>Strategia di verifica<sub>G</sub> (Testing)</b>                                                                            | <b>11</b> |
| 3.1      | Analisi statica <sub>G</sub> . . . . .                                                                                        | 11        |
| 3.1.1    | Checklist <sub>G</sub> per la verifica <sub>G</sub> (Analisi statica <sub>G</sub> tramite inspection <sub>G</sub> ) . . . . . | 11        |
| 3.2      | Analisi dinamica <sub>G</sub> . . . . .                                                                                       | 13        |
| <b>4</b> | <b>Testing - Fase RTB</b>                                                                                                     | <b>14</b> |
| 4.0.1    | Test di unità <sub>G</sub> . . . . .                                                                                          | 14        |
| 4.0.2    | Test di integrazione <sub>G</sub> . . . . .                                                                                   | 14        |
| 4.0.3    | Test di sistema <sub>G</sub> . . . . .                                                                                        | 15        |
| <b>5</b> | <b>Selezione dei modelli AI - Fase PB</b>                                                                                     | <b>16</b> |
| 5.1      | Applicazione delle metriche . . . . .                                                                                         | 16        |
| 5.1.1    | Analisi del testo con i Sei Cappelli per pensare di De Bono . . . . .                                                         | 16        |
| 5.1.2    | Traduzione del testo . . . . .                                                                                                | 17        |
| 5.1.3    | Correzione grammaticale del testo . . . . .                                                                                   | 17        |
| 5.1.4    | Riassunto del testo . . . . .                                                                                                 | 18        |
| 5.1.5    | Riscrittura del testo . . . . .                                                                                               | 18        |
| 5.1.6    | Distant writing <sub>G</sub> . . . . .                                                                                        | 18        |
| <b>6</b> | <b>Unit testing - Fase PB</b>                                                                                                 | <b>19</b> |
| 6.1      | Backend . . . . .                                                                                                             | 19        |
| 6.1.1    | test_ai_strategies.py . . . . .                                                                                               | 19        |
| 6.1.2    | Casi limite . . . . .                                                                                                         | 21        |
| 6.1.3    | test_app.py . . . . .                                                                                                         | 22        |
| 6.1.4    | test_text_cleaning.py . . . . .                                                                                               | 24        |
| 6.1.5    | test_model_strategies.py . . . . .                                                                                            | 24        |
| 6.1.6    | test_operation_registry.py . . . . .                                                                                          | 25        |
| <b>7</b> | <b>Test d'integrazione - Fase PB</b>                                                                                          | <b>27</b> |

|           |                                                  |           |
|-----------|--------------------------------------------------|-----------|
| <b>8</b>  | <b>Component Testing - Fase PB</b>               | <b>28</b> |
| <b>9</b>  | <b>End-to-End Testing (E2E) - Fase PB</b>        | <b>29</b> |
| <b>10</b> | <b>Cruscotto di valutazione della qualità</b>    | <b>30</b> |
| 10.1      | Qualità di processo . . . . .                    | 30        |
| 10.1.1    | Estimated at Completion (EAC) . . . . .          | 30        |
| 10.1.2    | Planned Value (PV) e Earned Value (EV) . . . . . | 31        |
| 10.1.3    | Schedule Performance Index (SPI) . . . . .       | 32        |
| 10.1.4    | Cost Performance Index (CPI) . . . . .           | 33        |
| 10.1.5    | Cost Variance (CV) . . . . .                     | 34        |
| 10.1.6    | Requirements Stability Index (RSI) . . . . .     | 35        |
| 10.1.7    | Rischi Non Calcolati (RNC) . . . . .             | 36        |
| 10.1.8    | Metriche Soddisfatte (MS) . . . . .              | 37        |

# 1 Introduzione

## 1.1 Scopo del documento

Il presente documento ha lo scopo di definire la strategia, le metodologie e le risorse pianificate per garantire la qualità del prodotto software ”**Second Brain**” e l’efficienza dei processi di sviluppo adottati dal gruppo. Esso funge da riferimento principale per le attività di **verifica<sub>G</sub>** (accertamento che il prodotto sia costruito correttamente) e **validazione<sub>G</sub>** (accertamento che il prodotto giusto sia stato costruito), stabilendo gli standard qualitativi che ogni artefatto deve soddisfare prima di essere approvato. In questo contesto, il termine ”**Qualità**” viene inteso secondo due prospettive complementari:

- **Qualità di processo:** L’aderenza del team al metodo di lavoro definito nelle *Norme di Progetto<sub>G</sub>*, basato sul miglioramento continuo e sull’efficienza delle procedure di sviluppo e test.
- **Qualità di prodotto:** La capacità del software di soddisfare i requisiti funzionali<sub>G</sub> e non funzionali esplicitati nell’*Analisi dei Requisiti<sub>G</sub>*.

Il documento è destinato al committente Prof. Vardanega e Prof. Cardin, all’azienda proponente Zucchetti S.p.A. e al gruppo di sviluppo ByteMe.

## 1.2 Scopo del prodotto

Il progetto **Second Brain**, proposto dall’azienda Zucchetti, mira a sviluppare un editor di testo<sub>G</sub> avanzato basato sul linguaggio Markdown<sub>G</sub> e potenziato dall’integrazione di Large Language Models (LLM). L’applicazione web consentirà all’utente di:

- Scrivere e visualizzare in tempo reale testi formattati in Markdown<sub>G</sub>;
- Interagire con un LLM per generare, riassumere, riscrivere, tradurre e analizzare testi;
- Sfruttare il metodo dei ”Sei cappelli per pensare” di Edward De Bono per ottenere analisi critiche multi-prospettiche;
- Fornire prompt<sub>G</sub> testuali per delegare all’AI la stesura completa del contenuto (*Distant Writing<sub>G</sub>*).

L’obiettivo è creare uno strumento di supporto alla scrittura creativa e al note-taking avanzato, esplorando le potenzialità pratiche degli LLM nel flusso di lavoro quotidiano.

## 1.3 Glossario<sub>G</sub>

Per evitare ambiguità, i termini tecnici usati nel documento vengono definiti in modo più approfondito nel documento Glossario<sub>G</sub> v2.0.0 che si può trovare nel sito web del gruppo ByteMe alla sezione Documenti Interni e raggiungibile dal seguente link:

ByteMe/Glossario<sub>G</sub> v2.0.0

I termini che vengono considerati meritevoli di approfondimenti sono indicati con una G a pedice.

## 1.4 Riferimenti

### 1.4.1 Riferimenti normativi

- *Norme di Progetto<sub>G</sub>* versione 2.0.0  
ByteMe/Norme di Progetto<sub>G</sub> v2.0.0

- Capitolato d'appalto C6: *Second Brain* (Zucchetti S.p.A.)  
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
- Regolamento del progetto didattico:  
<https://www.math.unipd.it/~tullio/IS-1/2023/Dispense/PD2.pdf>

#### 1.4.2 Riferimenti informativi

- *Analisi dei Requisiti<sub>G</sub>* versione 2.0.0 (Ultimo accesso: 01/04/2026)  
PB - Analisi dei Requisiti<sub>G</sub> v2.0.0
- Verballi interni ed esterni
- Glossario<sub>G</sub> v2.0.0 (Ultimo accesso: 01/04/2026)  
Documenti interni - Glossario<sub>G</sub> v2.0.0
- **ISO/IEC 12207:** *Systems and software engineering — Software life cycle processes*. (Standard di riferimento per il ciclo di vita del software).  
(Ultimo accesso: 20/02/2026)
- **ISO/IEC 25010:** *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE)*. (Modello di riferimento per la qualità del prodotto software).  
(Ultimo accesso: 16/02/2026)
- **Slide del corso di Ingegneria del Software:**
  - T9 - Verifica<sub>G</sub> e validazione<sub>G</sub>.
  - T10 - Analisi statica<sub>G</sub>.
  - T11 - Analisi dinamica<sub>G</sub>.
- **Slide e dispense del corso opzionale:** *Metodi e Tecnologie per lo Sviluppo Software*.
- **Documentazione tecnica strumenti:** *GitHub<sub>G</sub> Actions Documentation* (Ultimo accesso: 14/01/2026)

## 2 Metriche di qualità

### 2.1 Qualità di processo

#### 2.1.1 Processi primari di fornitura<sub>G</sub>

In questa sezione vengono definite le metriche relative alla gestione manageriale ed economica del progetto. L'obiettivo è monitorare l'allocazione delle risorse, il rispetto delle scadenze temporali e l'aderenza al preventivo definito in fase di candidatura. Per garantire un controllo oggettivo sull'avanzamento dei lavori, il gruppo adotta lo standard **EVA (Earned Value Analysis)<sub>G</sub>**, che permette di integrare la misurazione di tempi, costi e valore prodotto in un unico framework quantitativo.

#### Budget at Completion<sub>G</sub> (BAC)

Questo valore indica il budget totale massimo a disposizione del gruppo per lo svolgimento del progetto; rappresenta il limite di spesa totale.

Il preventivo iniziale, calcolato in fase di candidatura, ammonta a **11.395 euro** per un totale di 558 ore di lavoro.

- **Soglia accettabile:** minimizzato.
- **Soglia ottimale:** minimizzato.

#### Planned Value<sub>G</sub> (PV)

Valore monetario (o in ore) delle attività pianificate che avrebbero dovuto essere completate entro la data corrente. Si distingue in:

- **Sprint<sub>G</sub> Planned Value (SPV)**  
Valore pianificato per un singolo Sprint<sub>G</sub>. Calcolato come somma delle ore ruolo (OR) preventivate moltiplicate per il costo orario (CR).

| Calcolo                     | Soglia accettabile | Soglia ottimale |
|-----------------------------|--------------------|-----------------|
| $SPV = \sum (OR \times CR)$ | $> 0$              | $> 0$           |

Tabella 2: Parametro Sprint<sub>G</sub> Planned Value (SPV)

- **Project Planned Value (PPV)**  
Valore pianificato cumulativo per l'intero progetto (somma degli SPV passati).

| Calcolo                    | Soglia accettabile | Soglia ottimale     |
|----------------------------|--------------------|---------------------|
| $PPV = \sum SPV_{passati}$ | $> 0$              | $> 0$<br>$\leq BAC$ |

Tabella 3: Parametro Project Planned Value (PPV)

#### Actual Cost<sub>G</sub> (AC)

Costo effettivo sostenuto (ore lavorate  $\times$  costo orario) per il lavoro svolto finora. Si distingue in:

- **Sprint<sub>G</sub> Actual Cost (SAC)**  
Costo sostenuto nel singolo Sprint<sub>G</sub>.

| Calcolo                                            | Soglia accettabile | Soglia ottimale |
|----------------------------------------------------|--------------------|-----------------|
| $SAC = \sum \text{costi sostenuti nello Sprint}_G$ | $\leq SPV + 10\%$  | $\leq SPV$      |

Tabella 4: Parametro Sprint<sub>G</sub> Actual Cost (SAC)

- **Project Actual Cost (PAC)**

Costo totale sostenuto dall'inizio del progetto.

| Calcolo                      | Soglia accettabile | Soglia ottimale |
|------------------------------|--------------------|-----------------|
| $PAC = \sum_{s \in S} SAC_s$ | $\leq BAC$         | $\leq BAC$      |

Tabella 5: Parametro Project Actual Cost (PAC)

### Earned Value<sub>G</sub> (EV)

Rappresenta il valore del lavoro effettivamente completato e verificato alla data corrente. Per calcolarlo si adotta la tecnica di misurazione fisica **0/100<sub>G</sub>**: il valore pianificato di un'attività viene accreditato solo se l'attività è completata al 100%.

- **Sprint<sub>G</sub> Earned Value (SEV)**

Valore guadagnato nel singolo Sprint<sub>G</sub>. È la somma del Planned Value delle sole attività completate secondo la Definition of Done.

| Calcolo                      | Soglia accettabile  | Soglia ottimale |
|------------------------------|---------------------|-----------------|
| $SEV = \sum PV_{completati}$ | $\geq 90\%$ del SPV | $= SPV$         |

Tabella 6: Parametro Sprint<sub>G</sub> Earned Value (SEV)

- **Project Earned Value (PEV)**

Valore guadagnato cumulativo dall'inizio del progetto.

| Calcolo                    | Soglia accettabile  | Soglia ottimale |
|----------------------------|---------------------|-----------------|
| $PEV = \sum SEV_{passati}$ | $\geq 90\%$ del PPV | $= PPV$         |

Tabella 7: Parametro Project Earned Value (PEV)

### Schedule Variance<sub>G</sub> (SV)

Misura lo scostamento temporale del progetto rispetto alla pianificazione iniziale. Indica se il progetto è in anticipo, in pari o in ritardo in termini di valore prodotto.

| Calcolo        | Soglia accettabile     | Soglia ottimale |
|----------------|------------------------|-----------------|
| $SV = EV - PV$ | $SV \geq -10\%$ del PV | $SV \geq 0$     |

Tabella 8: Metrica Schedule Variance (SV)

**Cost Variance<sub>G</sub> (CV)**

Misura l'efficienza economica del progetto, confrontando il valore prodotto (guadagnato) con i costi effettivamente sostenuti. Indica se stiamo spendendo più del necessario per il lavoro svolto.

| Calcolo        | Soglia accettabile            | Soglia ottimale |
|----------------|-------------------------------|-----------------|
| $CV = EV - AC$ | $CV \geq -8\% \text{ del EV}$ | $CV \geq 0$     |

Tabella 9: Metrica Cost Variance (CV)

**Cost Performance Index<sub>G</sub> (CPI)**

Indice di efficienza dei costi. Rapporto tra il valore guadagnato e il costo effettivo sostenuto. Un valore inferiore a 1 indica che il costo sostenuto è superiore al valore prodotto.

| Calcolo               | Soglia accettabile | Soglia ottimale |
|-----------------------|--------------------|-----------------|
| $CPI = \frac{PV}{AC}$ | $CPI \geq 0.95$    | $CPI \geq 1.00$ |

Tabella 10: Metrica Cost Performance Index (CPI)

**Schedule Performance Index<sub>G</sub> (SPI)**

Indice di efficienza temporale; è il rapporto tra il valore guadagnato e il valore pianificato. Indica la velocità di avanzamento rispetto al piano.

| Calcolo               | Soglia accettabile | Soglia ottimale |
|-----------------------|--------------------|-----------------|
| $SPI = \frac{EV}{PV}$ | $SPI \geq 0.90$    | $SPI \geq 1.00$ |

Tabella 11: Metrica Schedule Performance Index (SPI)

**Estimated at Completion<sub>G</sub> (EAC)**

Stima aggiornata del costo totale finale del progetto, calcolata proiettando l'efficienza attuale (CPI) fino alla fine del progetto.

| Calcolo                 | Soglia accettabile | Soglia ottimale |
|-------------------------|--------------------|-----------------|
| $EAC = \frac{BAC}{CPI}$ | $EAC \leq BAC$     | $EAC \leq BAC$  |

Tabella 12: Metrica Estimated at Completion (EAC)

### 2.1.2 Processi primari di sviluppo<sub>G</sub>

Questa sezione raggruppa le metriche utilizzate per valutare la qualità delle attività di realizzazione tecnica del prodotto. Il monitoraggio si focalizza sulla **stabilità dei requisiti**, per controllare l'evoluzione dello scopo del progetto ed evitare cambiamenti incontrollati, e sulla **qualità strutturale del codice**, analizzando la complessità e la dimensione del software prodotto per garantirne la manutenibilità futura.

#### Requirements Stability Index<sub>G</sub> (RSI)

Misura la stabilità dei requisiti nel corso del tempo, indicando la percentuale di requisiti rimasti invariati rispetto al totale. Un valore basso indica frequenti cambiamenti nello scopo del progetto.

| Calcolo                                                   | Soglia accettabile | Soglia ottimale |
|-----------------------------------------------------------|--------------------|-----------------|
| $RSI = (1 - \frac{N_{modifiche}}{N_{totali}}) \times 100$ | $RSI \geq 80\%$    | $RSI = 100\%$   |

Tabella 13: Metrica Requirements Stability Index (RSI)

#### Copertura dei Requisiti<sub>G</sub> (CR)

Percentuale di requisiti implementati e verificati (soddisfatti) rispetto al totale pianificato, suddivisi per importanza (Obbligatori, Desiderabili, Opzionali).

| Calcolo                                             | Soglia accettabile                                  | Soglia ottimale                        |
|-----------------------------------------------------|-----------------------------------------------------|----------------------------------------|
| $C = \frac{N_{soddisfatti}}{N_{totali}} \times 100$ | Obbl: 100%<br>Desid: $\geq 40\%$<br>Opz: $\geq 0\%$ | Obbl: 100%<br>Desid: 100%<br>Opz: 100% |

Tabella 14: Metrica Copertura dei Requisiti (CR)

#### Lines of Code<sub>G</sub> (LOC)

Misura la dimensione fisica del software tramite il conteggio delle righe di codice sorgente (esclusi commenti e righe vuote). Questa è una metrica informativa di monitoraggio, quindi non vengono valutate le soglie.

| Calcolo                               | Soglia accettabile | Soglia ottimale |
|---------------------------------------|--------------------|-----------------|
| Somma righe logiche nei file sorgente | N/A                | N/A             |

Tabella 15: Metrica Lines of Code (LOC)

#### Complessità Ciclomatica<sub>G</sub> (McCabe)

Misura la complessità del flusso di controllo di un metodo o funzione calcolando il numero di cammini linearmente indipendenti. Una complessità elevata indica codice difficile da testare e mantenere.

| Calcolo             | Soglia accettabile | Soglia ottimale |
|---------------------|--------------------|-----------------|
| $v(G) = E - N + 2P$ | $\leq 8$           | $\leq 6$        |

Tabella 16: Metrica Complessità Ciclomatica (McCabe)

### 2.1.3 Processi di supporto<sub>G</sub>: documentazione e qualità

Le metriche esposte in questa sezione valutano l'efficacia<sub>G</sub> delle attività trasversali che supportano il ciclo di vita del software. L'attenzione è posta sulla **qualità della documentazione**, assicurando che sia leggibile, corretta e priva di ambiguità, e sull'efficacia<sub>G</sub> del **processo di verifica<sub>G</sub>**, misurando la capacità del gruppo di identificare e risolvere rischi e difetti in modo proattivo.

#### Indice di Gulpease<sub>G</sub> (IG)

Valuta la leggibilità di un testo redatto in lingua italiana per garantire che la documentazione sia comprensibile ai membri del gruppo e agli stakeholder esterni.

| Calcolo                                                                     | Soglia accettabile | Soglia ottimale |
|-----------------------------------------------------------------------------|--------------------|-----------------|
| $IG = 89 + \frac{300 \times N_{frasi} - 10 \times N_{lettere}}{N_{parole}}$ | $G \geq 40$        | $G \geq 60$     |

Tabella 17: Metrica Indice di Gulpease (IG)

#### Correttezza Ortografica (CO)

Misura il numero di errori ortografici o grammaticali residui nei documenti individuati durante i processi di verifica<sub>G</sub> o revisione.

| Calcolo                       | Soglia accettabile | Soglia ottimale |
|-------------------------------|--------------------|-----------------|
| Conteggio errori manuale/tool | 0                  | 0               |

Tabella 18: Metrica Correttezza Ortografica (CO)

#### Rischi Non Calcolati (RNC)

Quantità di rischi che si verificano durante il progetto ma che non erano stati previsti o analizzati in fase di candidatura e nel documento Piano di Progetto.

| Calcolo                     | Soglia accettabile | Soglia ottimale |
|-----------------------------|--------------------|-----------------|
| Conteggio rischi imprevisti | $\leq 2$           | 0               |

Tabella 19: Metrica Rischi Non Calcolati (RNC)

#### Metriche Soddisfatte (MS)

Percentuale di metriche (tra tutte quelle definite nel presente documento) che rientrano nelle soglie di accettabilità ad ogni verifica<sub>G</sub> di periodo. Indica lo stato di salute generale del progetto.

| Calcolo                                          | Soglia accettabile | Soglia ottimale |
|--------------------------------------------------|--------------------|-----------------|
| $MS = \frac{metricheOK}{TOTmetriche} \times 100$ | $\geq 75\%$        | 100%            |

Tabella 20: Metrica Metriche Soddisfatte (MS)

## 2.2 Qualità di prodotto

La qualità di prodotto nel software indica quanto bene un sistema riesce a soddisfare i bisogni degli utenti e gli obiettivi per cui è stato creato. Non significa solo che il programma funzioni, ma anche che sia affidabile, veloce, facile da usare e capace di essere migliorato nel tempo. Un software di buona qualità è quindi utile, stabile e adatto a evolversi insieme alle esigenze degli utenti.

### 2.2.1 Adeguatezza funzionale<sub>G</sub>

Indica quanto il software realizza correttamente tutte le funzioni richieste per soddisfare gli obiettivi dell'utente.

| Metrica                   | Scopo della metrica                                                                       | Soglia accettabile | Soglia ottimale  |
|---------------------------|-------------------------------------------------------------------------------------------|--------------------|------------------|
| Requirement coverage      | Verifica <sub>G</sub> che tutte le funzionalità principali dell'editor siano implementate | 90%                | 98%              |
| Acceptance test pass rate | Controlla che le funzionalità principali funzionino correttamente                         | 95%                | 99%              |
| Functional defect density | Misura quanti bug ci sono per funzione principale                                         | 0.5 bug/funzione   | 0.1 bug/funzione |

Tabella 21: Soglie metriche funzionalità del prodotto

### 2.2.2 Qualità dell'output generativo<sub>G</sub>

Misura quanto le risposte prodotte dal sistema sono corrette, coerenti con le istruzioni fornite e prive di contenuti inventati, garantendo l'affidabilità del supporto alla scrittura.

| Metrica                                             | Scopo della metrica                                                                                                | Soglia accettabile           | Soglia ottimale              |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|------------------------------|------------------------------|
| Aderenza al ruolo (System prompt <sub>GG</sub> )    | Valuta la capacità dell'IA di mantenere lo stile, il tono e il comportamento definiti nelle istruzioni di sistema. | $\geq 80\%$<br>(Punteggio 2) | $\geq 95\%$<br>(Punteggio 2) |
| Conformità strutturale (User prompt <sub>GG</sub> ) | Misura il rispetto dei vincoli e dei task specifici richiesti dall'utente nel prompt <sub>G</sub> operativo.       | $\geq 80\%$                  | $\geq 95\%$                  |
| Hallucination rate (Assenza di allucinazioni)       | Misura la frequenza di contenuti inventati o non deducibili dal testo di input fornito dall'utente.                | $\leq 5\%$                   | $< 1\%$                      |
| Completion success rate                             | Percentuale di suggerimenti AI accettati dall'utente                                                               | 75%                          | 90%                          |

Tabella 22: Soglie metriche qualità output AI

**Metodologia di calcolo** Per garantire l’oggettività delle misurazioni su output non deterministici, il calcolo delle metriche sopra riportate avviene secondo le seguenti formule:

- **Aderenza al ruolo (AR):** Rapporto percentuale tra il numero di generazioni che hanno ottenuto il punteggio massimo di aderenza (2) su un campione  $N$  di test.

$$AR = \frac{\text{N. generazioni con punteggio 2}}{N} \times 100$$

- **Conformità strutturale (CS):** Rapporto tra il numero di istruzioni atomiche rispettate ( $I_{ok}$ ) e il numero totale di istruzioni fornite ( $I_{tot}$ ) nei prompt<sub>G</sub> del campione.

$$CS = \frac{\sum I_{ok}}{\sum I_{tot}} \times 100$$

- **Tasso di allucinazione (HR):** Percentuale di risposte che presentano almeno un dato inventato rispetto al totale delle generazioni campionate ( $N$ ).

$$HR = \frac{\text{N. generazioni con allucinazioni}}{N} \times 100$$

**Nota:** Per non appesantire la stesura di questo documento, le procedure operative, le modalità di campionamento e le tecniche di valutazione manuale utilizzate per il calcolo empirico di queste metriche sono state scorporate e sono approfondite all’interno del documento **Norme di Progetto<sub>G</sub>**.

### 2.2.3 Affidabilità<sub>G</sub>

Rappresenta la capacità del software di funzionare in modo stabile e continuo senza guasti frequenti.

| Metrica      | Scopo della metrica                                                       | Soglia accettabile | Soglia ottimale |
|--------------|---------------------------------------------------------------------------|--------------------|-----------------|
| Availability | Percentuale di tempo in cui il software è operativo e stabile             | 99%                | 99.9%           |
| Error Rate   | Percentuale di operazioni che falliscono (salvataggio, apertura file, AI) | 1%                 | 0.1%            |
| MTTR         | Tempo medio per ripristinare il software dopo un crash                    | 60 minuti          | 10 minuti       |

Tabella 23: Soglie metriche affidabilità del prodotto

### 2.2.4 Efficienza

Indica quanto il sistema è veloce e quanto bene utilizza le risorse disponibili.

| <b>Metrica</b>                         | <b>Scopo della metrica</b>                                                                                                                     | <b>Soglia accettabile</b> | <b>Soglia ottimale</b> |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|------------------------|
| Response time<br>(Latenza interfaccia) | Tempo medio che l'interfaccia impiega per aggiornarsi correttamente                                                                            | 500 ms                    | 100 ms                 |
| AI processing time                     | Tempo intercorso tra l'invio del prompt <sub>G</sub> e la ricezione completa della risposta dall'API <sub>G</sub> (per il 95% delle richieste) | 60 sec                    | 30 sec                 |

Tabella 24: Soglie metriche efficienza del prodotto

### 2.2.5 Usabilità<sub>G</sub>

Descrive quanto il software è facile da imparare e da usare per gli utenti.

| <b>Metrica</b>         | <b>Scopo della metrica</b>                                                                     | <b>Soglia accettabile</b> | <b>Soglia ottimale</b> |
|------------------------|------------------------------------------------------------------------------------------------|---------------------------|------------------------|
| Task success rate      | Percentuale di utenti che completano correttamente un compito (scrittura, salvataggio, uso AI) | 80%                       | 95%                    |
| SUS Score              | Valutazione dell'usabilità tramite questionario standard                                       | 68                        | 85                     |
| User satisfaction rate | Percentuale di utenti soddisfatti delle funzioni AI                                            | 75%                       | 90%                    |

Tabella 25: Soglie metriche usabilità del prodotto

### 2.2.6 Manutenibilità<sub>G</sub>

Indica quanto è semplice correggere, modificare ed evolvere il software nel tempo.

| <b>Metrica</b>      | <b>Scopo della metrica</b>                                  | <b>Soglia accettabile</b> | <b>Soglia ottimale</b> |
|---------------------|-------------------------------------------------------------|---------------------------|------------------------|
| Test coverage       | Percentuale di codice coperto dai test (funzioni base + AI) | 70%                       | 90%                    |
| Change failure rate | Percentuale di modifiche che introducono problemi o bug     | 15%                       | 5%                     |

Tabella 26: Soglie metriche manutenibilità del prodotto

### 3 Strategia di verifica<sub>G</sub> (Testing)

#### 3.1 Analisi statica<sub>G</sub>

L'analisi statica<sub>G</sub> consiste nella verifica<sub>G</sub> dei documenti e del codice senza esecuzione del software, con l'obiettivo di individuare errori sintattici, incongruenze, violazioni degli standard e problemi strutturali.

Nel progetto Second Brain, l'analisi statica<sub>G</sub> viene effettuata principalmente tramite attività di inspection, supportate da checklist<sub>G</sub> condivise, che guidano i verificatori nel controllo sistematico della qualità della documentazione e del codice.

##### 3.1.1 Checklist<sub>G</sub> per la verifica<sub>G</sub> (Analisi statica<sub>G</sub> tramite inspection<sub>G</sub>)

Questa sezione contiene le liste di controllo che i Verificatori devono utilizzare obbligatoriamente prima di approvare un documento o un pezzo di codice. L'obiettivo è standardizzare il controllo qualità ed evitare errori ricorrenti. Questa checklist<sub>G</sub> viene aggiornata progressivamente dai verificatori e permette di evitare gli errori più comuni e semplici da individuare.

| Oggetto                    | Errore                           | Azione correttiva                                                                                                                                                                       |
|----------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Immagini e tabelle         | Mancanza di didascalia (caption) | Ogni tabella o immagine deve avere una didascalia esplicativa numerata.                                                                                                                 |
| Struttura                  | Sezioni vuote o orfane           | Rimuovere o riempire le sezioni che contengono solo il titolo senza testo.                                                                                                              |
| Glossario <sub>G</sub>     | Termine non marcato o errato     | Verificare che i termini del glossario <sub>G</sub> siano segnati con la "G" a pedice e rispettino il case sensitivity.                                                                 |
| Ortografia                 | Errori di battitura o sintassi   | Utilizzare il correttore automatico, verificare personalmente i testi, rimuovere refusi e doppi spazi.                                                                                  |
| Percorsi File              | Path immagini errato             | Verificare che i percorsi siano relativi e corretti (es: <code>../UCimg/diagramma.jpg</code> per i diagrammi, <code>../logo.jpg</code> per i loghi) per evitare errori di compilazione. |
| Versionamento <sub>G</sub> | Disallineamento versione         | Controllare che la versione indicata nel frontespizio coincida con l'ultima voce del registro delle modifiche.                                                                          |
| Versionamento <sub>G</sub> | Nome file senza versione         | Il nome del documento deve contenere il numero di versione corrente (es. <code>Nome_doc.v1.0.0</code> ).                                                                                |

Tabella 27: Checklist<sub>G</sub> documentazione (LaTeX)

| Oggetto          | Errore                    | Azione correttiva                                                                                                                                                    |
|------------------|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sincronizzazione | Push senza pull           | Eseguire sempre <code>git<sub>G</sub> pull</code> prima di <code>git<sub>G</sub> push</code> per risolvere conflitti locali ed evitare merge <sub>G</sub> complessi. |
| Pulizia          | Codice commentato o morto | Rimuovere blocchi di codice commentato inutile o funzioni non utilizzate prima del push.                                                                             |

Tabella 28: Checklist<sub>G</sub> codice e push (Git<sub>G</sub>)

| Oggetto      | Errore                              | Azione correttiva                                                                                                           |
|--------------|-------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Tracciamento | Requisiti per un caso d'uso assenti | Ad ogni caso d'uso deve corrispondere almeno un requisito.                                                                  |
| Diagrammi    | Diagrammi dei casi d'uso erranei    | I diagrammi devono riportare la corretta numerazione e nomenclatura del caso d'uso.                                         |
| Tracciamento | Caso d'uso senza requisiti          | Ogni Caso d'Uso (UC) deve generare almeno un Requisito.                                                                     |
| UML          | Inconsistenza diagrammi             | La numerazione e i nomi negli schemi UML devono corrispondere esattamente al testo descrittivo.                             |
| Attori       | Attori non definiti                 | Verificare che tutti gli attori (es. Utente, Sistema AI <sub>G</sub> ) siano descritti nella sezione introduttiva degli UC. |
| Nomenclatura | Codice identificativo errato        | Ogni requisito deve avere un codice univoco che segue la nomenclatura standard definita.                                    |

Tabella 29: Checklist<sub>G</sub> Analisi Dei Requisiti<sub>G</sub>

### 3.2 Analisi dinamica<sub>G</sub>

L'analisi dinamica<sub>G</sub> del software *Second Brain* viene condotta attraverso l'esecuzione di test strutturati su diversi livelli di granularità, secondo il modello a V del ciclo di vita del software. L'obiettivo è verificare che il sistema soddisfi i requisiti funzionali<sub>G</sub> e non funzionali, garantendo affidabilità, correttezza e usabilità del prodotto finale.

Le attività di testing sono organizzate nei seguenti livelli:

- **Unit Testing:** verifica<sub>G</sub> il corretto funzionamento delle singole unità software (funzioni o classi) in isolamento. L'obiettivo è individuare errori logici e garantire che ogni unità si comporti come previsto.
- **Test di Integrazione<sub>G</sub>:** verifica<sub>G</sub> l'interazione tra più componenti del sistema. In questa fase si controlla che i moduli collaborino correttamente tra loro e che lo scambio di dati avvenga senza errori.
- **Component Testing:** si concentra sulla validazione<sub>G</sub> di componenti funzionali completi, considerati come unità logiche più ampie rispetto alle singole classi. Permette di verificare il comportamento di parti significative del sistema in condizioni realistiche.
- **End-to-End Testing (E2E):** verifica<sub>G</sub> il comportamento dell'intero sistema simulando scenari d'uso reali. L'obiettivo è assicurare che il flusso completo, dalla richiesta dell'utente fino alla risposta finale, funzioni correttamente in un ambiente il più possibile simile a quello di produzione.

Questa struttura consente di individuare errori in modo progressivo, partendo dai singoli componenti fino ad arrivare alla validazione<sub>G</sub> dell'intero sistema.

## 4 Testing - Fase RTB

**Priorità di testing** Durante lo sviluppo del Proof of Concept (PoC), l'analisi dinamica<sub>G</sub> si concentra sulla verifica<sub>G</sub> della fattibilità tecnologica e sull'integrazione tra i componenti. Le priorità sono così definite:

- **Alta:** Test di sistema<sub>G</sub> (funzionalità AI core) e test di integrazione<sub>G</sub> (API<sub>G</sub> Auth + AI, Backend-Database<sub>G</sub>).
- **Media:** Test di unità (logiche isolate come `auth.py` o formattazione dei prompt<sub>G</sub>).

L'obiettivo attuale non è raggiungere una *Code Coverage* esaustiva (che sarà centrale nella fase PB), ma validare che i rischi tecnologici siano stati mitigati.

### 4.0.1 Test di unità<sub>G</sub>

**Obiettivo:** Verificare il corretto funzionamento delle singole unità software in isolamento (funzioni, metodi, moduli), assicurando che producano l'output atteso per ogni input fornito.

| Test ID | Funzione testata                        | Descrizione                                                     | Input                       | Output atteso                           |
|---------|-----------------------------------------|-----------------------------------------------------------------|-----------------------------|-----------------------------------------|
| TU-1    | <code>genera_hash()</code>              | Verifica <sub>G</sub> generazione hash PBKDF2 con salt casuale. | pwd: "test123"              | Stringa formato <code>salt\$hash</code> |
| TU-2    | <code>verifica<sub>G</sub>_pwd()</code> | Verifica <sub>G</sub> confronto con una password corretta.      | pwd: "test123", hash valido | True                                    |
| TU-3    | <code>verifica<sub>G</sub>_pwd()</code> | Verifica <sub>G</sub> rifiuto con una password errata.          | pwd: "wrong", hash valido   | False                                   |
| TU-4    | <code>verifica<sub>G</sub>_pwd()</code> | Gestione delle eccezioni per hash malformati o nulli.           | hash malformato             | False                                   |

Tabella 30: Test di unità

### 4.0.2 Test di integrazione<sub>G</sub>

**Obiettivo:** Verificare che i diversi moduli del sistema (Frontend, Backend, Database<sub>G</sub> e LLM esterni) comunichino correttamente tra loro attraverso le rispettive interfacce e API<sub>G</sub>.

| Test ID | Descrizione                                                                                                                                                                                                                                                                                                       | Requisito |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| TI-1    | <b>Health Check Database<sub>G</sub>:</b> Verifica <sub>G</sub> che il backend riesca a stabilire una connessione stabile con il database <sub>G</sub> PostgreSQL <sub>G</sub> interrogando l'endpoint <sub>G</sub> dedicato ( <code>/api<sub>G</sub>/test-db-connection</code> ) ed eseguendo una query di test. | ROP-F-22  |
| TI-2    | <b>Health Check LLM:</b> Verifica <sub>G</sub> in fase di avvio dell'applicazione che le variabili d'ambiente (API <sub>G</sub> Key e Base URL) siano correttamente caricate e che il client verso il provider esterno venga inizializzato con successo.                                                          | RO-V-13   |
| TI-3    | Verifica <sub>G</sub> che la chiamata di registrazione dal backend salvi correttamente e persistentemente il record del nuovo utente all'interno del database <sub>G</sub> PostgreSQL <sub>G</sub> .                                                                                                              | ROP-F-11  |
| TI-4    | Verifica <sub>G</sub> che l'endpoint <sub>G</sub> del backend formatti correttamente il prompt <sub>G</sub> richiesto e lo inoltri con successo alle API <sub>G</sub> del provider LLM, ricevendo una risposta valida.                                                                                            | RO-V-13   |
| TI-5    | Verifica <sub>G</sub> che la funzione frontend <code>loginUtente()</code> componga correttamente il body JSON della richiesta HTTP POST e gestisca adeguatamente la risposta del backend (es. token di sessione o errore).                                                                                        | ROP-F-9   |

Tabella 31: Test di integrazione<sub>G</sub>

#### 4.0.3 Test di sistema<sub>G</sub>

**Obiettivo:** Validare il comportamento del software nel suo complesso, simulando i percorsi d'uso (End-to-End) dell'utente finale tramite l'interfaccia grafica per garantire la copertura dei casi d'uso.

| Test ID | Descrizione                                                                                                                                                                                                                                       | Caso d'Uso (UC) |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------|
| TS-1    | <b>Flusso autenticazione:</b> L'utente accede alla pagina di login, inserisce credenziali valide e preme "Accedi". Il sistema lo reindirizza all'editor, nasconde i bottoni di login/registrazione e l'area utente.                               | UC-5.0, UC-5.5  |
| TS-2    | <b>Flusso AI core:</b> L'utente carica un file locale, ne seleziona una porzione, richiede un'operazione all'AI (es. "Riassumi"). Il sistema elabora la richiesta e inietta il risultato nel pannello "Storico Generazioni".                      | UC-2.*          |
| TS-3    | <b>Flusso editor e I/O:</b> L'utente scrive testo formattato in Markdown <sub>G</sub> nell'editor, verifica <sub>G</sub> che la sintassi venga riconosciuta, ed esegue il salvataggio (download) del file in formato .md sul proprio dispositivo. | UC-1.0, UC-4.0  |

Tabella 32: Test di sistema<sub>G</sub>

## 5 Selezione dei modelli AI - Fase PB

Il sistema è progettato per utilizzare modelli di intelligenza artificiale differenti in base alla funzionalità richiesta dall'utente (ad esempio riassunto, riscrittura, traduzione o analisi). Questa scelta consente di ottimizzare le prestazioni, sfruttando modelli più adatti a specifici compiti: modelli più deterministici per operazioni analitiche e modelli più creativi per attività di generazione o espansione del testo. Per valutare quale modello fosse migliore abbiamo preso in considerazione diverse metriche, in seguito elencate nella tabella.

| Metrica                     | Descrizione                                                                               | Metodo di valutazione                                                                                      | Valore atteso                |
|-----------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------------------------|
| Tempo di risposta           | Latenza totale per la generazione dell'output.                                            | Misurazione automatica del tempo tra invio <i>prompt<sub>G</sub></i> e ricezione risposta.                 | $\leq 60$ s                  |
| Aderenza al ruolo (AR)      | Capacità di rispettare il tono e il comportamento del <i>system prompt<sub>GG</sub></i> . | Valutazione manuale su scala discreta (0, 1, 2) su un campione di test.                                    | $\geq 80\%$<br>(Punteggio 2) |
| Conformità strutturale (CS) | Rispetto dei vincoli atomici definiti nello <i>user prompt<sub>GG</sub></i> .             | Analisi binaria del rapporto tra istruzioni rispettate e istruzioni totali.                                | $\geq 80\%$                  |
| Tasso di allucinazione (HR) | Frequenza di informazioni inventate o non deducibili dall'input.                          | Verifica <sub>G</sub> manuale di corrispondenza con il testo sorgente ( <i>ground truth<sub>G</sub></i> ). | $\leq 5\%$                   |

Tabella 33: Elenco metriche per selezione modelli AI

### 5.1 Applicazione delle metriche

Al fine di garantire una valutazione esaustiva, ogni membro del gruppo ha condotto sessioni di test comparativi individuali, redigendo appositi resoconti tecnici sulle prestazioni dei modelli linguistici. Di seguito si riporta la sintesi dei risultati emersi dai test collettivi.

#### 5.1.1 Analisi del testo con i Sei Cappelli per pensare di De Bono

A titolo esemplificativo, si riporta il resoconto decisionale relativo all'implementazione della funzionalità *Sei Cappelli di De Bono*, con focus sul **Cappello Bianco**.

Il *system prompt<sub>GG</sub>* del Cappello Bianco richiede al modello di agire in modo puramente oggettivo e imparziale. Il task impone di estrarre fatti e dati ignorando qualsiasi opinione o emozione, restituendo il risultato strettamente in formato Markdown<sub>G</sub>. Per garantire la massima analiticità, la temperatura<sub>G</sub> di generazione è stata fissata a 0.1.

**Confronto modelli e risultati** I test si sono concentrati principalmente sul confronto tra **Llama 3.2<sub>G</sub> (3B)** e **Gemma 3<sub>G</sub> (4B / 27B)**, mentre altri modelli sono stati scartati per tempo di risposta troppo alto. Lo studio ha prodotto i seguenti esiti:

- **Aderenza al ruolo:** Il modello Llama 3.2<sub>G</sub> ha mostrato una tendenza a "uscire dal personaggio", inserendo occasionalmente un tono discorsivo o frasi di circostanza (es. "*Ecco l'analisi richiesta...*") non compatibili con la freddezza richiesta dal ruolo (punteggio AR prevalentemente pari a 1). Al contrario, Gemma 3<sub>G</sub> ha dimostrato un'aderenza al ruolo

eccellente (punteggio 2), limitandosi a eseguire un'anatomia del testo con totale freddezza e distacco emotivo.

- **Conformità strutturale:** Entrambi i modelli hanno rispettato rigorosamente le istruzioni dello user prompt<sub>GG</sub>.
- **Tasso di allucinazione:** Entrambi i modelli hanno registrato tassi ampiamente sotto la soglia critica del 5%, basandosi quasi esclusivamente sul testo fornito (basso *Groundedness error*), confermando la bontà del parametro di temperatura<sub>G</sub> basso (0.1).

**Modello scelto** Sulla base dei dati raccolti, il modello **Gemma 3<sub>G</sub>** ha superato ampiamente le soglie ottimali per le metriche AR e CS in compiti di profilazione psicologica complessa. Di conseguenza, il gruppo ha deciso di assegnare l'intera suite dei Sei Cappelli al modello Gemma 3<sub>G</sub>, riservando Llama 3.2<sub>G</sub> a operazioni editoriali più standard (riassunti, correzioni grammaticali) dove il tono discorsivo risulta meno penalizzante. Sono stati testati anche gli altri cinque Cappelli di De Bono e i risultati sono coerenti con quanto riportato per il Cappello Bianco.

### 5.1.2 Traduzione del testo

### 5.1.3 Correzione grammaticale del testo

Per integrare lo studio sui modelli è stata condotta una sessione di testing dedicata alle capacità di correzione grammaticale, analizzando come la scala dei parametri influenzi la capacità di individuare refusi e incongruenze sintattiche.

**Confronto modelli e risultati** L'analisi ha messo in luce una netta distinzione tra le performance dei modelli in base alla loro densità computazionale, con particolare attenzione alla capacità di "comprendere" l'errore all'interno del contesto:

- **Qwen 3 (30B):** Ha restituito una qualità della correzione definita "perfetta". Il modello identifica con precisione chirurgica ogni anomalia testuale, tuttavia la velocità di caricamento e di elaborazione è risultata molto bassa, rendendolo meno agile<sub>G</sub> nei flussi di lavoro intensivi.
- **Gemma 3<sub>G</sub> (27B):** Si è distinto come il modello più equilibrato. Ha garantito correzioni impeccabili, dimostrando una comprensione del contesto della frase superiore ai modelli più piccoli, mantenendo al contempo una velocità di risposta (latenza) ottimale per l'integrazione operativa.
- **Gemma 3<sub>G</sub> (270M):** Nonostante la risposta quasi istantanea dovuta alle ridotte dimensioni, il modello si è rivelato inadeguato per il compito. Tende a tralasciare numerosi errori e non dimostra una comprensione logica della struttura della frase, limitandosi a una revisione superficiale e incompleta.

**Impatto della scala dei parametri** I test hanno confermato che per operazioni di correzione critica, la dimensione del modello è una variabile discriminante:

- **Modelli Small (es. 270M):** Risultano troppo scarsi per la produzione, con un alto tasso di errori residui.
- **Modelli Large (27B - 30B):** Mostrano una sensibilità linguistica elevata. La differenza tra il 27B e il 30B in termini di accuratezza è minima, ma il vantaggio del modello da 27B in termini di velocità operativa è significativo.

**Modello scelto** Sulla base delle evidenze raccolte, il gruppo ha selezionato Gemma 3<sub>G</sub> (27B) come motore principale per le funzioni di correzione. Il modello ha superato la soglia di affidabilità richiesta, garantendo "correzioni perfette" senza i rallentamenti strutturali riscontrati nel modello Qwen da 30B.

#### **5.1.4 Riassunto del testo**

#### **5.1.5 Riscrittura del testo**

#### **5.1.6 Distant writing<sub>G</sub>**

## 6 Unit testing - Fase PB

### 6.1 Backend

Per l'implementazione dei test unitari del backend è stato utilizzato **Pytest**, uno dei framework di testing più diffusi nell'ecosistema Python. Pytest consente di scrivere test in modo semplice, leggibile e scalabile, riducendo al minimo il codice boilerplate rispetto ad approcci più tradizionali.

Uno dei principali vantaggi di Pytest è la sua sintassi intuitiva: i test possono essere scritti come semplici funzioni Python, senza la necessità di creare classi o utilizzare strutture complesse.

Pytest offre anche funzionalità avanzate come la parametrizzazione, utile per eseguire lo stesso test su più casi di input.

La scelta di utilizzare Pytest è quindi motivata dalla sua flessibilità, dalla leggibilità dei test prodotti e dall'ampio supporto della comunità.

#### 6.1.1 test\_ai\_strategies.py

Questa sezione descrive i test relativi al modulo `ai_strategies`, che definisce le strategie di costruzione dei `promptG` utilizzati per interagire con i modelli di intelligenza artificiale.

Il modulo fornisce diverse strategie, ciascuna progettata per uno specifico tipo di operazione (ad esempio riassunto, traduzione o analisi secondo i Sei Cappelli di De Bono). Ogni strategia è responsabile della generazione dei `promptG` di sistema e utente, oltre alla definizione di una `temperaturaG` adeguata al tipo di elaborazione.

I test verificano la corretta costruzione dei `promptG`, la coerenza delle configurazioni, la validità delle strategie registrate e il comportamento del sistema in presenza di input non validi o casi limite.

**BaseAIStrategy** Questa sezione descrive i test relativi alla classe base `BaseAIStrategy`. Essendo una classe astratta, non è pensata per un utilizzo diretto, ma come fondamento per le strategie concrete. I test verificano che il comportamento previsto venga rispettato, garantendo così l'integrità dell'architettura.

| Codice | Descrizione                                                                                                                                                                                                                                                                                                         | Stato |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-0   | Il metodo <code>build()</code> di <code>BaseAIStrategy</code> lancia <code>NotImplementedError</code> apposta, per obbligare le sottoclassi a reimplementarlo. Il test verifica <sub>G</sub> che questo meccanismo funzioni: se qualcuno chiama <code>build()</code> sulla classe base, deve ottenere quell'errore. | Pass  |
| UT-1   | Verifica <sub>G</sub> che la classe base definisca una variabile di classe <code>temperature</code> di default ( <code>float</code> ), necessaria per le configurazioni di base.                                                                                                                                    | Pass  |

Tabella 34: Tabella unit test di `BaseAIStrategy`

**SimplePromptStrategy** In questa sezione vengono testate le funzionalità della classe `SimplePromptStrategy`, utilizzata per generare `promptG` di tipo semplice. I test si concentrano sulla correttezza della struttura dell'output e sulla presenza delle informazioni fondamentali (ruolo, task e testo), assicurando che il `promptG` venga costruito in modo coerente e conforme alle specifiche.

| Codice | Descrizione                                                                                                                                                                                        | Stato |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-2   | Verifica <sub>G</sub> la forma dell'output: <code>build()</code> deve restituire esattamente una tupla/lista di 2 elementi, entrambi stringhe.                                                     | Pass  |
| UT-3   | Verifica <sub>G</sub> che il role appaia effettivamente nel system prompt <sub>GG</sub> .                                                                                                          | Pass  |
| UT-4   | Verifica <sub>G</sub> che la frase fissa di comportamento, quella che dice all'AI di non fare preamboli, sia sempre presente nel system prompt <sub>GG</sub> , indipendentemente dal ruolo scelto. | Pass  |
| UT-5   | Verifica <sub>G</sub> che il task appaia nello user prompt.                                                                                                                                        | Pass  |
| UT-6   | Verifica <sub>G</sub> che il testo passato a <code>build()</code> finisca nello user prompt <sub>GG</sub> .                                                                                        | Pass  |
| UT-7   | Verifica <sub>G</sub> che tutte le funzionalità di scrittura hanno role e task corretti.                                                                                                           | Pass  |

Tabella 35: Tabella unit tests di SimplePromptStrategy

**TestTranslationStrategy** Questa sezione raccoglie i test per la strategia di traduzione. L'obiettivo è verificare che il sistema generi prompt<sub>G</sub> corretti per diverse lingue, mantenendo una struttura uniforme e includendo correttamente sia il testo di input sia la lingua di destinazione.

| Codice | Descrizione                                                                                                                                    | Stato |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-8   | Verifica <sub>G</sub> la forma dell'output: <code>build()</code> deve restituire esattamente una tupla/lista di 2 elementi, entrambi stringhe. | Pass  |
| UT-9   | Verifica <sub>G</sub> che la lingua appaia effettivamente nello user prompt <sub>GG</sub> .                                                    | Pass  |
| UT-10  | Verifica <sub>G</sub> che il testo passato a <code>build()</code> finisca nello user prompt <sub>GG</sub> .                                    | Pass  |
| UT-11  | Verifica <sub>G</sub> che il system prompt <sub>GG</sub> sia uguale per tutte le lingue.                                                       | Pass  |
| UT-12  | Verifica <sub>G</sub> che tutte le lingue supportate funzionino correttamente.                                                                 | Pass  |

Tabella 36: Tabella unit tests per la traduzione del testo

**TestDeBonoHatStrategy** Qui vengono descritti i test relativi alla strategia basata sui “Sei Cappelli per Pensare”. I test verificano che ogni configurazione produca un prompt<sub>G</sub> coerente, includendo correttamente la temperatura specifica per ogni cappello, lo user prompt<sub>GG</sub> e il system prompt<sub>GG</sub> e il testo fornito, oltre a garantire il funzionamento per tutte le varianti previste.

| Codice | Descrizione                                                                                                                                    | Stato |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-13  | Verifica <sub>G</sub> la forma dell'output: <code>build()</code> deve restituire esattamente una tupla/lista di 2 elementi, entrambi stringhe. | Pass  |
| UT-14  | Verifica <sub>G</sub> che il cappello selezionato appaia effettivamente nel system prompt <sub>GG</sub> .                                      | Pass  |
| UT-15  | Verifica <sub>G</sub> che il focus appaia effettivamente nello user prompt <sub>GG</sub> .                                                     | Pass  |
| UT-16  | Verifica <sub>G</sub> che il testo passato a <code>build()</code> finisca nello user prompt <sub>GG</sub> .                                    | Pass  |
| UT-17  | Verifica <sub>G</sub> che tutti i 6 cappelli funzionino correttamente.                                                                         | Pass  |

Tabella 37: Tabella unit tests per i Sei Cappelli di De Bono

**TestStrategiesDict** Questa sezione valida la correttezza del dizionario che raccoglie tutte le strategie disponibili. I test controllano sia la completezza delle strategie registrate sia la loro tipologia, assicurando che ogni voce sia associata alla classe corretta e produca output validi.

| Codice | Descrizione                                                                                 | Stato |
|--------|---------------------------------------------------------------------------------------------|-------|
| UT-18  | Verifica <sub>G</sub> che tutte le strategies attese siano presenti                         | Pass  |
| UT-19  | Verifica <sub>G</sub> che tutti i valori siano istanze di BaseAIStrategy                    | Pass  |
| UT-20  | Verifica <sub>G</sub> che tutte le strategies producano due stringhe non vuote.             | Pass  |
| UT-21  | Verifica <sub>G</sub> che le funzionalità di scrittura sono istanze di SimplePromptStrategy | Pass  |
| UT-22  | Verifica <sub>G</sub> che le traduzioni sono istanze di TranslationStrategy                 | Pass  |
| UT-23  | Verifica <sub>G</sub> che i cappelli sono istanze di DeBonoHatStrategy                      | Pass  |

Tabella 38: Tabella unit tests di TestStrategiesDict

### 6.1.2 Casi limite

In questa sezione vengono analizzati i comportamenti del sistema in condizioni limite o non standard. L'obiettivo è verificare la robustezza delle strategie rispetto a input insoliti, garantendo stabilità e prevedibilità anche in situazioni limite.

**Stringa vuota** Questa sottosezione verifica<sub>G</sub> il comportamento delle strategie quando ricevono una stringa vuota in input. I test assicurano che il sistema non generi errori e continui a produrre output validi.

| Codice | Descrizione                                                                       | Stato |
|--------|-----------------------------------------------------------------------------------|-------|
| UT-24  | Verifica <sub>G</sub> SimplePromptStrategy accetti la stringa vuota senza errori. | Pass  |
| UT-25  | Verifica <sub>G</sub> TranslationStrategy accetti la stringa vuota senza errori.  | Pass  |
| UT-26  | Verifica <sub>G</sub> DeBonoHatStrategy accetti la stringa vuota senza errori.    | Pass  |

Tabella 39: Tabella unit tests per casi limite (stringa vuota)

**Solo spazi o testo molto lungo** In questa sottosezione vengono testati input particolari come stringhe composte solo da spazi e testi molto lunghi. L'obiettivo è garantire che il sistema gestisca correttamente sia casi minimali sia input di grandi dimensioni.

| Codice | Descrizione                                                                          | Stato |
|--------|--------------------------------------------------------------------------------------|-------|
| UT-27  | Verifica <sub>G</sub> che SimplePromptStrategy accetti input con solo spazi.         | Pass  |
| UT-28  | Verifica <sub>G</sub> che SimplePromptStrategy accetti testi molto lunghi (> 10000). | Pass  |
| UT-29  | Verifica <sub>G</sub> che DeBonoHatStrategy gestisca newline e tab nell'input.       | Pass  |

Tabella 40: Tabella unit tests per casi limite (solo spazi o testo lungo)

**Caratteri speciali e unicode** Questa sottosezione verifica<sub>G</sub> la capacità del sistema di gestire caratteri non standard, come emoji, simboli speciali e caratteri non latini. I test assicurano la compatibilità con input internazionali e complessi.

| Codice | Descrizione                                                                                 | Stato |
|--------|---------------------------------------------------------------------------------------------|-------|
| UT-30  | Verifica <sub>G</sub> che TranslationStrategy gestisca emoji e simboli speciali nell'input. | Pass  |
| UT-31  | Verifica <sub>G</sub> che TranslationStrategy gestisca caratteri non latini nell'input.     | Pass  |

Tabella 41: Tabella unit tests per casi limite (caratteri speciali)

**None come input** Questa sottosezione analizza il comportamento del sistema quando viene passato un valore None. I test sono progettati per verificare che venga sollevato un errore appropriato, evidenziando eventuali mancanze nella gestione degli input non validi.

| Codice | Descrizione                                                                       | Stato |
|--------|-----------------------------------------------------------------------------------|-------|
| UT-32  | Verifica <sub>G</sub> che SimplePromptStrategy lanci TypeError se l'input è None. | Pass  |
| UT-33  | Verifica <sub>G</sub> che TranslationStrategy lanci TypeError se l'input è None.  | Pass  |
| UT-34  | Verifica <sub>G</sub> che DeBonoHatStrategy lanci TypeError se l'input è None.    | Pass  |

Tabella 42: Tabella unit tests per casi limite (None come input)

**Costruttore con valori vuoti** In questa sottosezione vengono testati i costruttori delle diverse strategie con parametri vuoti. L'obiettivo è verificare che il sistema sia sufficientemente flessibile da accettare configurazioni minime senza generare errori.

| Codice | Descrizione                                                                 | Stato |
|--------|-----------------------------------------------------------------------------|-------|
| UT-35  | Verifica <sub>G</sub> che SimplePromptStrategy accetti role e task vuoti.   | Pass  |
| UT-36  | Verifica <sub>G</sub> che TranslationStrategy accetti la lingua vuota.      | Pass  |
| UT-37  | Verifica <sub>G</sub> che DeBonoHatStrategy accetti hat_name e focus vuoti. | Pass  |

Tabella 43: Tabella unit tests per casi limite (costruttore con valori vuoti)

### 6.1.3 test\_app.py

Questa sezione descrive i test relativi al modulo app, che rappresenta il backend dell'applicazione basato su Flask<sub>G</sub>.

Il modulo espone un endpoint<sub>G</sub> API<sub>G</sub> per la generazione di contenuti tramite modelli di intelligenza artificiale. La richiesta viene elaborata selezionando l'operazione desiderata, costruendo i prompt<sub>G</sub> tramite le strategie configurate e invocando il modello appropriato.

I test verificano il corretto funzionamento dell'endpoint<sub>G</sub>, la validazione<sub>G</sub> degli input, la gestione degli errori e la coerenza del formato delle risposte.

**TestGenerateInputValidation** Questa sezione descrive i test volti a verificare la corretta validazione<sub>G</sub> degli input forniti all'endpoint<sub>G</sub> di generazione. L'obiettivo è garantire che il sistema gestisca in modo appropriato richieste incomplete, malformate o non conformi alle specifiche, restituendo codici di errore adeguati e messaggi informativi.

| Codice | Descrizione                                                                                                                                     | Stato |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-38  | Verifica <sub>G</sub> che l'assenza del campo obbligatorio text produca un errore HTTP 400 e un messaggio di errore nel campo 'generated_text'. | Pass  |
| UT-39  | Verifica <sub>G</sub> che una stringa vuota " " restituisca 400 come il caso precedente.                                                        | Pass  |
| UT-40  | Verifica <sub>G</sub> la resilienza del sistema: se l'operazione è sconosciuta, deve eseguire il fallback sul default (summary).                | Pass  |

Tabella 44: Tabella unit tests per test\_app.py

**TestGenerateOperations** In questa sezione vengono presentati i test relativi al corretto funzionamento delle operazioni supportate dal sistema. Viene verificato che ogni operazione definita nel registry venga eseguita correttamente e che il sistema adotti comportamenti di default robusti in assenza di parametri espliciti.

| Codice | Descrizione                                                                                                                                                 | Stato |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-41  | Verifica <sub>G</sub> il corretto funzionamento di ogni operazione presente nel registry (summary, fix_grammar, rewrite, distant_writing) (parametrizzato). | Pass  |
| UT-42  | Verifica <sub>G</sub> che la richiesta vada a buon fine anche se il campo operation viene totalmente omesso dal payload.                                    | Pass  |

Tabella 45: Tabella unit tests per test\_app.py

**TestGenerateErrorHandling** I test descritti in questa sezione hanno l'obiettivo di valutare la robustezza del sistema nella gestione degli errori. Viene verificato che eventuali malfunzionamenti del modello AI siano correttamente intercettati, evitando impatti sulla disponibilità del servizio e garantendo una comunicazione chiara degli errori al client.

| Codice | Descrizione                                                                                                                              | Stato |
|--------|------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-43  | Verifica <sub>G</sub> che un crash del modello AI non causi un downtime del server, ma venga catturato e restituito come HTTP 500.       | Pass  |
| UT-44  | Verifica <sub>G</sub> che il messaggio d'errore inviato al frontend contenga dettagli tecnici utili al debugging (testo dell'eccezione). | Pass  |

Tabella 46: Tabella unit tests per test\_app.py

**TestResponseFormat** Questa sezione analizza la struttura e la coerenza del formato delle risposte restituite dal sistema. I test garantiscono che tutte le risposte, sia in caso di successo che di errore, rispettino un formato standardizzato, facilitando l'integrazione e l'elaborazione da parte dei client.

| Codice | Descrizione                                                                                                                                         | Stato |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-45  | Verifica <sub>G</sub> che la risposta di successo deve sempre contenere la chiave <code>generated_text</code> .                                     | Pass  |
| UT-46  | Verifica <sub>G</sub> che anche in caso di errore (400) il frontend riceva la chiave <code>generated_text</code> per mostrare il messaggio a video. | Pass  |
| UT-47  | Verifica <sub>G</sub> che l'header di risposta sia rigorosamente <code>application/json</code> , garantendo la compatibilità con i parser client.   | Pass  |

Tabella 47: Tabella unit tests per `test_app.py`

#### 6.1.4 `test_text_cleaning.py`

Questa sezione descrive i test relativi al modulo di utilità `text_cleaning`, responsabile della normalizzazione dell'output generato dall'intelligenza artificiale. I test assicurano che le funzioni basate su espressioni regolari puliscano correttamente il testo, rimuovendo prefissi colloquiali, spaziature errate e garantendo l'assenza di case-sensitivity.

| Codice | Descrizione                                                                                                                                        | Stato |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| UT-48  | Verifica <sub>G</sub> la rimozione di prefissi introduttivi tipici dei modelli AI (es. "Ecco un riassunto:", "La traduzione è:", ecc.).            | Pass  |
| UT-49  | Verifica <sub>G</sub> che la funzione rimuova correttamente eventuali andate a capo (newlines) o spaziature vuote all'inizio del testo restituito. | Pass  |
| UT-50  | Verifica <sub>G</sub> che la pulizia sia case-insensitive, intercettando correttamente i preamboli scritti in tutto maiuscolo o minuscolo.         | Pass  |

Tabella 48: Tabella unit tests per `test_text_cleaning.py`

#### 6.1.5 `test_model_strategies.py`

Questa sezione descrive i test relativi al modulo `model_strategies`, che gestisce l'utilizzo di diversi modelli di intelligenza artificiale attraverso un'interfaccia comune.

Il modulo è progettato per permettere l'uso di più modelli in modo uniforme, nascondendo i dettagli tecnici delle singole implementazioni. Inoltre, include un meccanismo di gestione delle istanze che evita la creazione duplicata degli stessi modelli.

I test hanno lo scopo di verificare il corretto funzionamento delle strategie, il comportamento del sistema di gestione delle istanze e la corretta gestione di errori e casi particolari.

| Codice | Descrizione                                                                                      | Stato |
|--------|--------------------------------------------------------------------------------------------------|-------|
| UT-51  | Verifica <sub>G</sub> che la classe base non possa essere usata direttamente.                    | Pass  |
| UT-52  | Verifica <sub>G</sub> che get_model restituisca sempre la stessa istanza (Singleton).            | Pass  |
| UT-53  | Verifica <sub>G</sub> che la strategia Llama chiami correttamente il client OpenAI.              | Pass  |
| UT-54  | Verifica <sub>G</sub> che la strategia Llama gestisca (o sollevi) errori dall'API <sub>G</sub> . | Pass  |
| UT-55  | Verifica <sub>G</sub> che get_model distingua correttamente tra tipi diversi di modelli.         | Pass  |
| UT-56  | Verifica <sub>G</sub> che le istanze create siano del tipo corretto e non si sovrappongano.      | Pass  |
| UT-57  | Verifica <sub>G</sub> che venga lanciato un errore se mancano le variabili d'ambiente.           | Pass  |
| UT-58  | Verifica <sub>G</sub> che la strategia gestisca una risposta vuota o mancante.                   | Pass  |
| UT-59  | Verifica <sub>G</sub> il corretto ripopolamento del registro interno se svuotato manualmente.    | Pass  |

Tabella 49: Tabella unit tests per test\_model\_strategies.py

#### 6.1.6 test\_operation\_registry.py

Questa sezione descrive i test relativi al modulo operation\_registry, che definisce la configurazione delle operazioni disponibili nel sistema. Il modulo associa a ciascuna operazione una coppia composta da:

- una strategia di modello AI, responsabile della generazione del testo;
- una strategia di prompt<sub>G</sub>, responsabile della costruzione delle istruzioni da fornire al modello.

Questa struttura consente di gestire in modo centralizzato e coerente le diverse funzionalità (ad esempio riassunto, traduzione o analisi), garantendo flessibilità nella scelta del modello e della logica di generazione.

I test verificano la correttezza del registro delle operazioni, la validità delle configurazioni associate e il comportamento del sistema in condizioni normali e in casi limite.

| <b>Codice</b> | <b>Descrizione</b>                                                                                                                               | <b>Stato</b> |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| UT-60         | Verifica <sub>G</sub> che il registro delle operazioni sia correttamente strutturato come un dizionario.                                         | Pass         |
| UT-61         | Accerta la presenza delle operazioni fondamentali (summary, fix_grammar, ecc.) nel registro.                                                     | Pass         |
| UT-62         | Verifica <sub>G</sub> che l'operazione di default esista nel registro.                                                                           | Pass         |
| UT-63         | Verifica <sub>G</sub> che ogni operazione abbia una classe modello valida e una strategia di prompt <sub>G</sub> corretta.                       | Pass         |
| UT-64         | Verifica <sub>G</sub> specifica che 'summary' usi la strategia di riassunto.                                                                     | Pass         |
| UT-65         | Verifica <sub>G</sub> specifica che 'fix grammar' usi la strategia di correzione grammaticale.                                                   | Pass         |
| UT-66         | Verifica <sub>G</sub> specifica che 'rewrite' usi la strategia di riscrittura.                                                                   | Pass         |
| UT-67         | Verifica <sub>G</sub> specifica che 'distant writing <sub>G</sub> ' generi un prompt <sub>G</sub> coerente.                                      | Pass         |
| UT-68         | Verifica <sub>G</sub> che nessuna operazione sia configurata a None.                                                                             | Pass         |
| UT-69         | Verifica <sub>G</sub> che ogni operazione usi un'istanza di strategia diversa.                                                                   | Pass         |
| UT-70         | Valida che tutte le chiavi del registro usino lo standard snake_case (minuscole e underscore).                                                   | Pass         |
| UT-71         | Con una simulazione di un input reale verifica <sub>G</sub> che ogni strategia nel registro sia in grado di generare prompt <sub>G</sub> validi. | Pass         |
| UT-72         | Verifica <sub>G</sub> che l'operazione di default abbia una config completa.                                                                     | Pass         |

Tabella 50: Tabella unit tests per test\_operation\_registry.py

## 7 Test d'integrazione - Fase PB

## 8 Component Testing - Fase PB

## 9 End-to-End Testing (E2E) - Fase PB

## 10 Cruscotto di valutazione della qualità

In questa sezione vengono presentati gli indicatori di qualità relativi al processo produttivo, al fine di valutarne l'andamento durante le due fasi di progetto: RTB (Requirements and Technology Baseline) e PB (Product Baseline).

### 10.1 Qualità di processo

#### 10.1.1 Estimated at Completion (EAC)

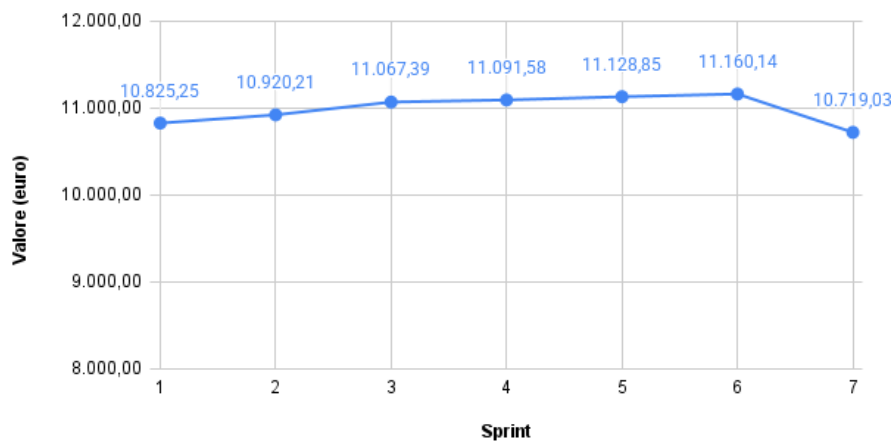


Figura 1: Andamento del valore di Estimated at Completion (EAC) nella fase RTB

**RTB:** L'analisi dell'Estimated at Completion (EAC) mostra un andamento che si è mantenuto costantemente al di sotto del preventivo totale iniziale (Budget at Completion, pari a 11.395 euro) per tutta la durata della fase. Nonostante alcune fluttuazioni iniziali e i rallentamenti temporali registrati negli Sprint<sub>G</sub> 3 e 5, l'ottima efficienza sui costi mantenuta dal team (CPI sempre  $\geq 1$ ) ha permesso di stabilizzare la stima finale verso il basso. Il gruppo chiude quindi la fase RTB con una proiezione economica solida, confermando l'assenza di rischi legati allo sforamento del budget assegnato.

### 10.1.2 Planned Value (PV) e Earned Value (EV)

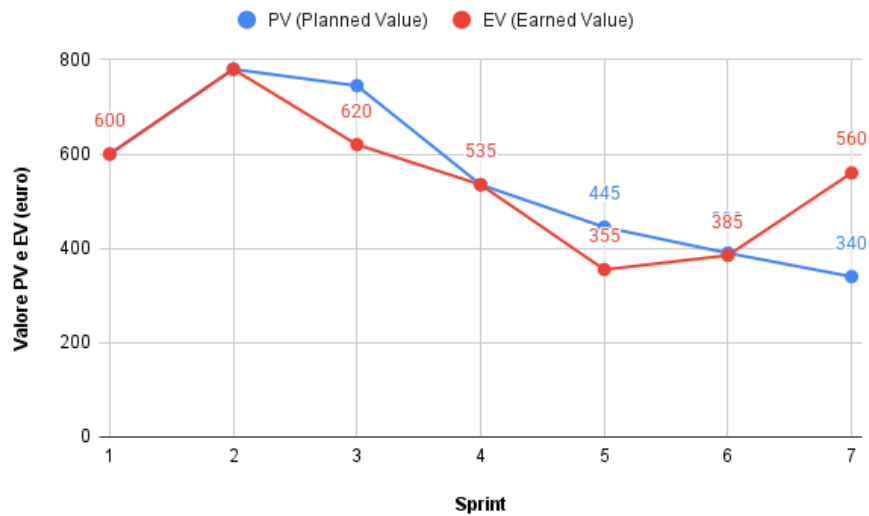


Figura 2: Andamento e confronto di PV e EV nella fase RTB

**RTB:** Dal confronto tra le curve di Planned Value (costo pianificato) ed Earned Value (valore prodotto), emerge una flessione dell'EV durante lo Sprint<sub>G</sub> 3 e lo Sprint<sub>G</sub> 5 a causa della ridotta disponibilità oraria durante il periodo natalizio e per gli esami invernali.

Tuttavia, analizzando anche l'Actual Cost (AC, non rappresentato in figura ma rendicontato nei consuntivi), emerge che i costi effettivi si sono mantenuti sempre allineati o inferiori all'EV: questo dimostra che le spese sono state costantemente sotto controllo e proporzionate al lavoro realmente svolto, senza alcuno spreco. Durante gli Sprint<sub>G</sub> 6 e 7 il team ha incrementato la produttività, riportando l'EV ad allinearsi al PV e chiudendo la fase senza debiti temporali e con un risparmio effettivo ( $AC < PV$ ).

### 10.1.3 Schedule Performance Index (SPI)

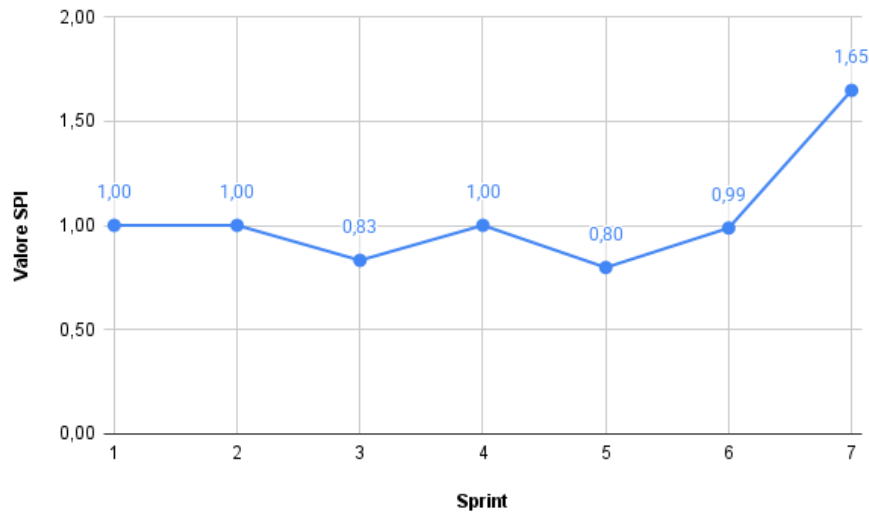


Figura 3: Andamento dello Schedule Performance Index (SPI) nella fase RTB

**RTB:** L'indice di efficienza della schedulazione (SPI) evidenzia visibilmente l'impatto della sessione d'esami, toccando un picco negativo di 0.83 nello Sprint<sub>G</sub> 3 e di 0.80 nello Sprint<sub>G</sub> 5. Questo scostamento si è tradotto in una Schedule Variance (SV) negativa, che ha toccato un picco di -125 € nello Sprint<sub>G</sub> 3.

Tuttavia, la redistribuzione dei carichi di lavoro ha permesso di recuperare il gap già negli Sprint<sub>G</sub> successivi (4 e 6) e di chiudere lo Sprint<sub>G</sub> 7 in perfetto orario (SPI a 1.65) e con un saldo temporale tornato in positivo. L'andamento a "V" del grafico dimostra un'ottima capacità di reazione del team agli imprevisti temporali. L'obiettivo del team è rendere l'indice più stabile nella prossima fase di PB attraverso una migliore pianificazione delle attività.

#### 10.1.4 Cost Performance Index (CPI)

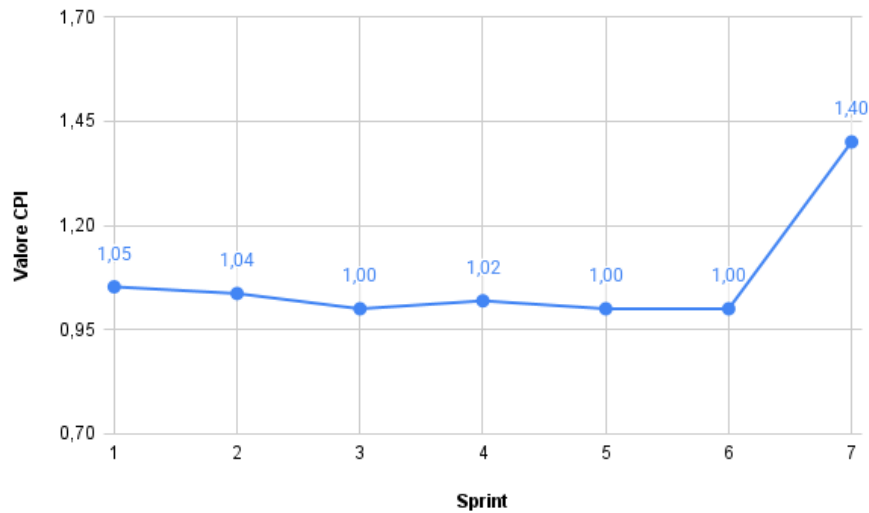


Figura 4: Andamento del Cost Performance Index (CPI) nella fase RTB

**RTB:** L'indice di efficienza dei costi (CPI) mostra un andamento eccellente, mantenendosi costantemente attorno al valore 1.0 o superiore. Ciò significa che per ogni euro speso dal progetto, il valore prodotto è stato pari o superiore a un euro. Nonostante il ritardo temporale ( $SPI < 1$ ) negli Sprint<sub>G</sub> 3 e 5, il gruppo è stato molto efficiente economicamente: le ore lavorate sono state produttive e non ci sono stati sprechi di risorse o rilavorazioni costose che avrebbero abbassato il CPI.

### 10.1.5 Cost Variance (CV)

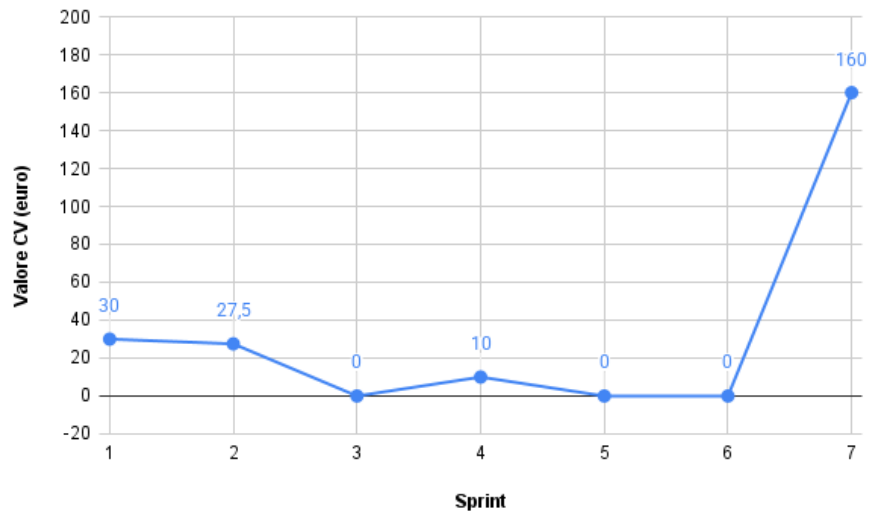


Figura 5: Andamento della Cost Variance (CV) nella fase RTB

**RTB:** La varianza dei costi (CV), calcolata come differenza tra Earned Value e Actual Cost ( $EV - AC$ ), si è mantenuta positiva o prossima allo zero per tutta la durata della fase. Questo indicatore conferma che il gruppo non ha mai speso più del valore generato. Al termine della RTB, il risparmio cumulato è pari a 227,50 €, garantendo al progetto un prezioso margine di sicurezza economico da investire nella successiva fase di Product Baseline.

### 10.1.6 Requirements Stability Index (RSI)

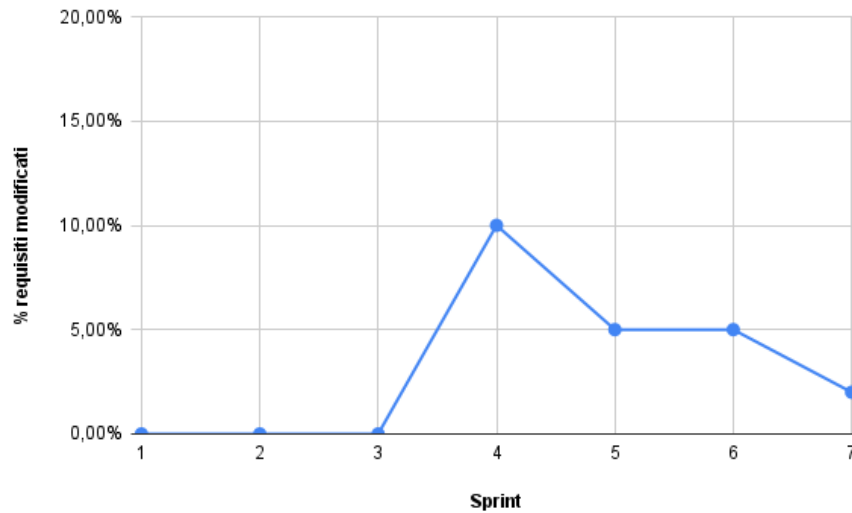


Figura 6: Andamento del Requirements Stability Index (RSI) nella fase RTB

**RTB:** L'indice di stabilità dei requisiti ha subito alcune variazioni in particolare durante lo Sprint<sub>G</sub> 4 e 5. A seguito del colloquio di verifica<sub>G</sub> con il prof. Cardin, il gruppo ha dovuto apportare correzioni e raffinamenti ai casi d'uso e ai requisiti precedentemente individuati. Negli ultimi sprint<sub>G</sub> (Sprint<sub>G</sub> 6), con il consolidamento dell'architettura per il PoC, l'indice si è stabilizzato, indicando che il perimetro del progetto è ora ben definito.

### 10.1.7 Rischi Non Calcolati (RNC)

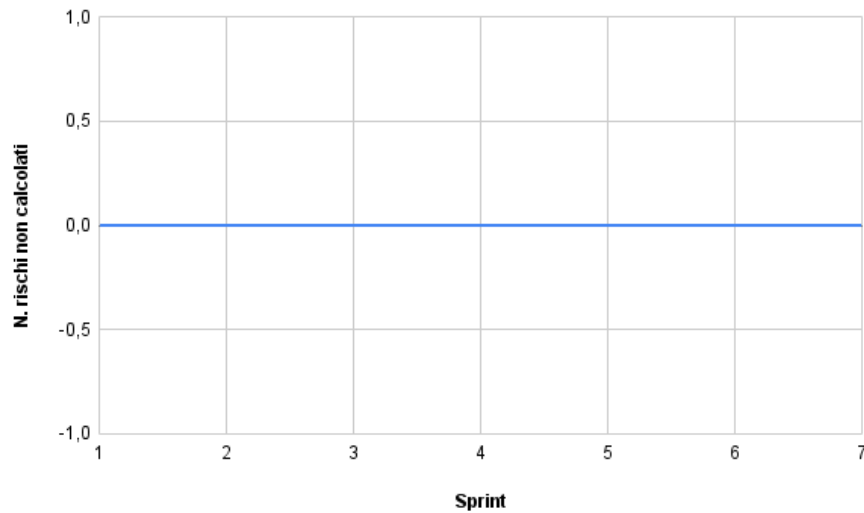


Figura 7: Numero di rischi non calcolati (RNC) nella fase RTB

**RTB:** Durante la fase RTB, il numero di rischi imprevisti è stato pari a zero (o molto basso). Le principali criticità riscontrate, come la ridotta disponibilità dovuta agli esami (RO-1) e le difficoltà di apprendimento delle tecnologie LLM (RT-1), erano state correttamente previste e mitigate. Il gruppo può ritenersi soddisfatto della capacità di previsione e gestione dei rischi.

### 10.1.8 Metriche Soddisfatte (MS)

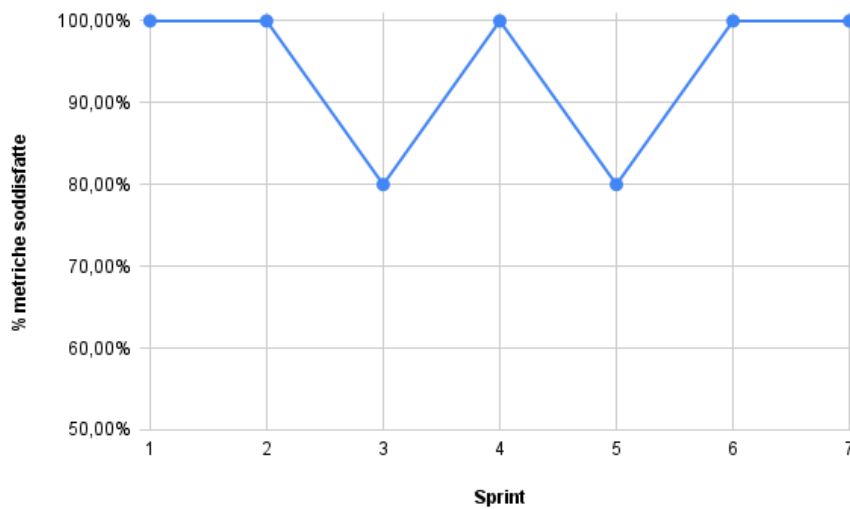


Figura 8: Percentuale delle metriche soddisfatte (MS) nella fase RTB

**RTB:** La percentuale complessiva di metriche soddisfatte ha registrato due flessioni (attorno all'80%) in corrispondenza degli Sprint<sub>G</sub> 3 e 5, principalmente a causa degli sforamenti temporali (SPI  $\geq$  0.90). Tuttavia, la tenuta impeccabile delle metriche economiche (CPI, CV sempre ottimali) e il recupero finale hanno permesso di chiudere la fase con il 100% degli indicatori in stato accettabile o ottimale. Il processo adottato è pertanto ritenuto maturo e validato.